ECE3411

Lecture 8

Task Based Programming & Real Time Clock

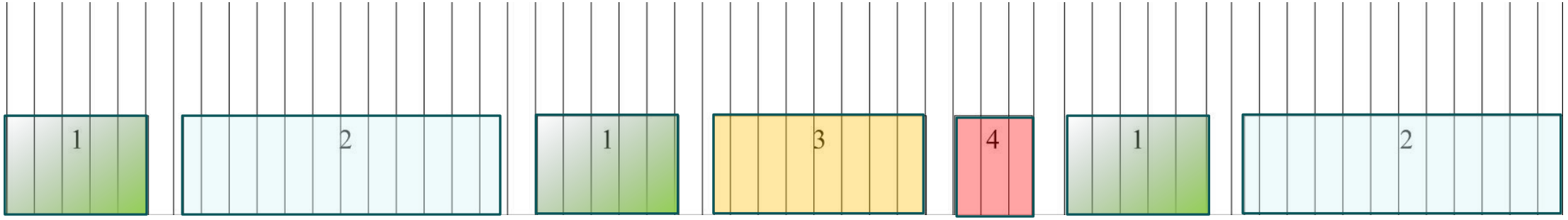
Sung Yeul Park
Department of Electrical & Computer Engineering
University of Connecticut
Email: sung_yeul.park@uconn.edu

The Importance of Organizing Code

- The organized and neat code is highly preferable to inefficient and confusing code.
- As a general rule of thumb, the more that you write, the more debugging you will need to do later.
- We also want to reduce the amount of code sitting in our ISRs because 1) it's more difficult to debug, 2) Long ISRs may keep interrupts disabled so long that the MCU misses other events.
- We also want to cram more features/peripherals into our programs without hurting our performance.
- How can we do this???

Task-Based Programming

- Suppose that we have, 1. ADC, 2. Math Function #1, 3. UART, 4. Math Function #2
- Each task takes a certain amount of time to finish, so it's more efficient if we periodically space them out and repeat them at specific intervals.



Task Based Programming

- Ex. According to the datasheet, after the initial conversion, ADC will take ~13 ADC Clock cycles. This gives you insight as to how much time the 'task' should take. Doing this estimation for all tasks can give us a rough idea of how to schedule all of our features.

- Suppose that each task, 1, 2, 3, and 4 have software timer variables,

```
int ADC_Timer
int Math_1_Timer
int UART_Timer
int Math_2_Timer
```

Task Based Programming

- Let's say that our ISR occurs every 1mS, ADC takes 2mS, Math_1 takes 4mS, UART takes 3mS, and Math_2 takes 5mS. Initializing our timers with 1mS in between them,

```
int ADC_Timer    = 18; // Waits 0mS
int Math_1_Timer = 21; // Waits 3mS
int UART_Timer   = 26; // Waits 8mS
int Math_2_Timer = 30; // Waits 12mS
```

- The whole cycle, repeating our four functions will take 18mS. With a 1mS ISR we decrement our variables at this frequency,

```
ISR(TIMERn_COMPN_Vect)
  {
       if(ADC_Timer > 0)        ADC_Timer--;
       if(Math_1_Timer > 0) Math_1_Timer--;
       if(UART_Timer > 0)     UART_Timer--;
       if(Math_2_Timer > 0) Math_2_Timer--;
  }
```

Task Based Programming

- while(1) loop can now keep track of the status of our timer/counters. Once they've been decremented to zero, we reset the timers and then call our functions. Since we've staggered them already, they can be reset to the same values. You can space them out whatever you like. However, remember that the timer ISR will keep counting during the execution of your functions, and you may have two timers at the same time!!

```
while(1)
{
        if(ADC_Timer == 0{
                ADC_Timer=18; ADC();
        }

        if(Math_1_Timer == 0){

        Math_1_Timer=18; Math1();
        }

        if(UART_Timer == 0){
                UART_Timer=18;
                        UART();
        }

        if(Math_2_Timer == 0){
                Math_2_Timer=18;
                Math2();
        }
}
```

Task Based Programming

- In summary, Task Based Programming allows us to organize and schedule many functions without reducing our performance.
- It also reduces the amount of code we keep in our timer ISR, and provides a basis for us to add things like tickets, variable scheduling times, permissions/priority (the basics of an operating system).

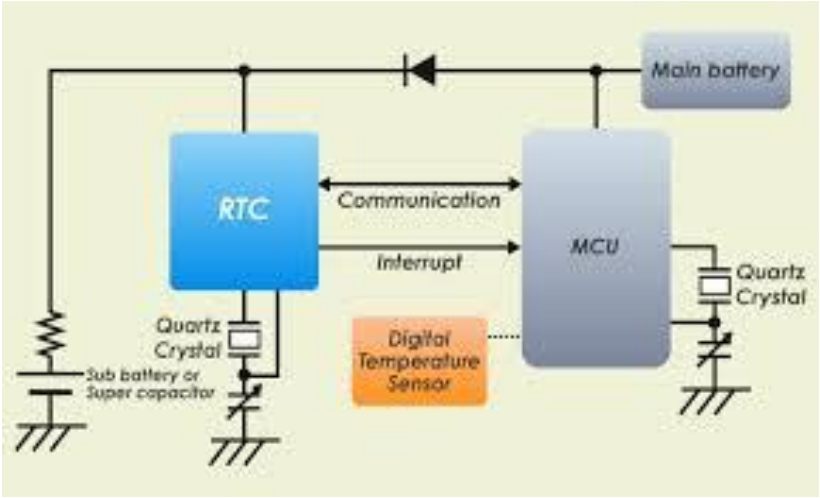
Task Based Programming Sample Code

```
....
int TaskTime = 500;
volatile int SWTaskTimer=TaskTime;

ISR(TIMER0_COMPA_vect)
{
  if (SWTaskTimer>0) {SWTaskTimer--;}
}

// 1ms ISR for Timer 0 assuming F_CPU = 1MHz
void InitTimer0(void)
{
  TCCR0A |= (1<<WGM01);
  OCR0A = 124;
  TIMSK0 =2;
  TCCR0B = 2; //Timer starts
}
....
```

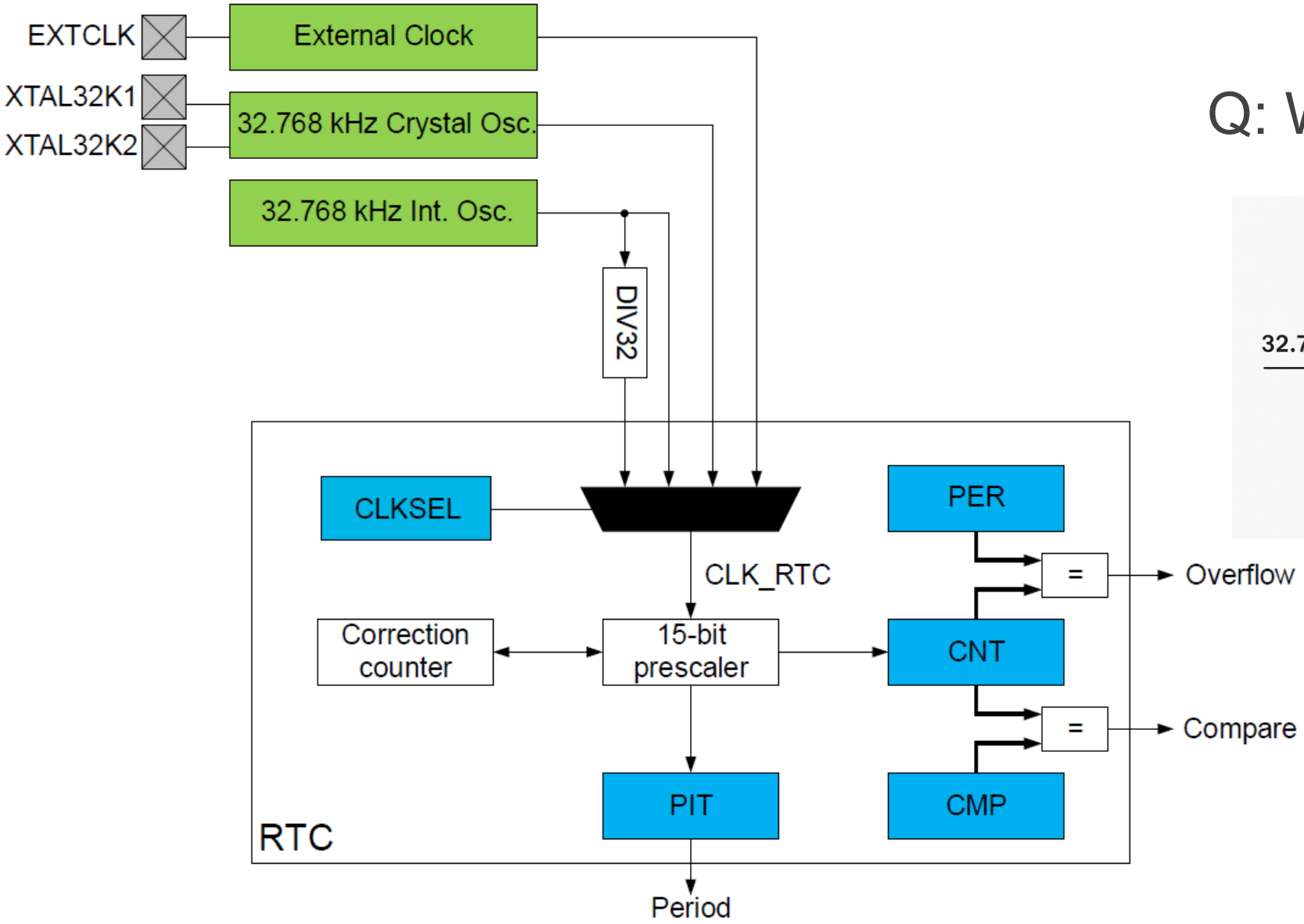
```
int main(void)
{
  …
  InitTimer0();
  …
  sei(); // Enable global interrupt

  while(1)
  {
    if (SWTaskTimer == 0)
    {
      Task();
      SWTaskTimer == TaskTime;
    }
  }

  return 0;
}
```

Introduction of RTC

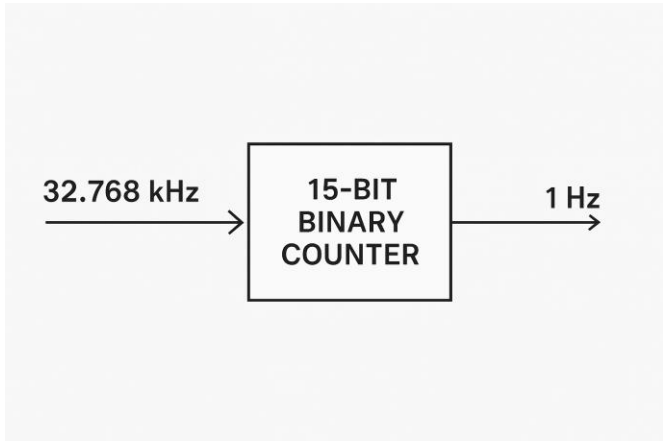
- **What is RTC?**
- RTC stands for Real-Time Counter (or Real-Time Clock).
- It keeps track of time even when the main processor is idle or in sleep mode.
- Useful for:
  - Time-based event triggering
  - Low-power periodic wake-up
- **RTC in AVR128DB48**
- 16-bit counter
- Can use internal 32.768kHz oscillator or external crystal
- Supports overflow interrupt
- Operates independently of CPU



RTC Block Diagram



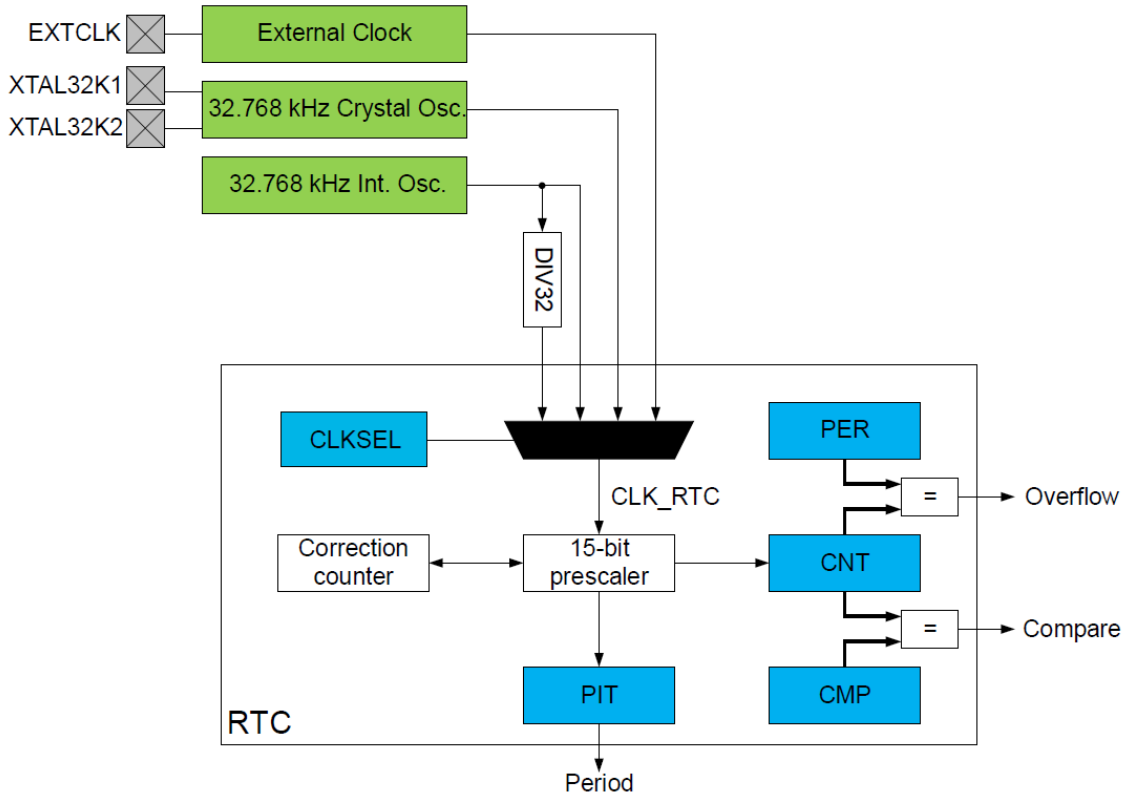
Q: Why 32.768 kHz ?



Register Overview and Configuration

Registers related to RTC

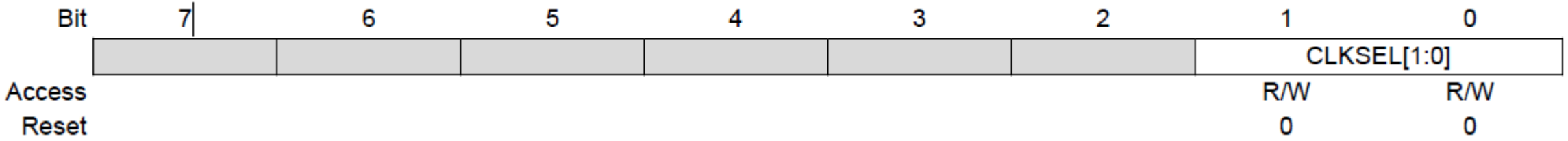
Register	Description
RTC.CTRLA	Enables RTC, sets prescaler
RTC.CLKSEL	Selects clock source (e.g., internal 32kHz)
RTC.PER	Sets period for overflow
RTC.CNT	Current counter value
RTC.INTCTRL	Enables interrupt
RTC.INTFLAGS	Interrupt flag status



Configuration Steps

Select clock source: RTC.CLKSEL → Set overflow period: RTC.PER → Enable overflow interrupt: RTC.INTCTRL → Enable RTC: RTC.CTRLA → Enable global interrupts: sei()

RTC.CLKSEL



Bits 1:0 – CLKSEL[1:0] Clock Select

Writing these bits select the source for the RTC clock (CLK_RTC).

Value	Name	Description
0x0	OSC32K	32.768 kHz from OSC32K
0x1	OSC1K	1.024 kHz from OSC32K
0x2	XOSC32K	32.768 kHz from XOSC32K
0x3	EXTCLK	External clock from the EXTCLK/ XTALHF1 pin

RTC.PER

The RTC.PERL and RTC.PERH register pair represents the 16-bit value, RTC.PER. The low byte [7:0] (suffix L) is accessible at the original offset. The high byte [15:8] (suffix H) can be accessed at offset + 0x01.
Due to the synchronization between the RTC clock and main clock domains, there is a latency of two RTC clock cycles from updating the register until this has an effect. The application software needs to check that the PERBUSY flag in RTC.STATUS is cleared before writing to this register.

Bit	15	14	13	12	11	10	9	8
	PER[15:8]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	1	1	1	1	1	1	1	1

Bit	7	6	5	4	3	2	1	0
	PER[7:0]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	1	1	1	1	1	1	1	1

Bits 15:8 – PER[15:8] Period High Byte
These bits hold the MSB of the 16-bit Period register.

Bits 7:0 – PER[7:0] Period Low Byte
These bits hold the LSB of the 16-bit Period register.

RTC.INTCTRL

Bit	7	6	5	4	3	2	1	0
							CMP	OVF
Access							R/W	R/W
Reset							0	0

Bit 1 – CMP Compare Match Interrupt Enable

Enable interrupt-on-compare match (that is, when the value from the Count (RTC.CNT) register matches the value from the Compare (RTC.CMP) register).

Value	Description
0	The compare match interrupt is disabled
1	The compare match interrupt is enabled

Bit 0 – OVF Overflow Interrupt Enable

Enable interrupt-on-counter overflow (that is, when the value from the Count (RTC.CNT) register matched the value from the Period (RTC.PER) register and wraps around to zero).

Value	Description
0	The overflow interrupt is disabled
1	The overflow interrupt is enabled

RTC.INTCTRL

Bit	7	6	5	4	3	2	1	0
							CMP	OVF
Access							R/W	R/W
Reset							0	0

Bit 1 – CMP Compare Match Interrupt Enable

Enable interrupt-on-compare match (that is, when the value from the Count (RTC.CNT) register matches the value from the Compare (RTC.CMP) register).

Value	Description
0	The compare match interrupt is disabled
1	The compare match interrupt is enabled

Bit 0 – OVF Overflow Interrupt Enable

Enable interrupt-on-counter overflow (that is, when the value from the Count (RTC.CNT) register matched the value from the Period (RTC.PER) register and wraps around to zero).

Value	Description
0	The overflow interrupt is disabled
1	The overflow interrupt is enabled

RTC Example Code

```
#include <avr/io.h>
#include <avr/interrupt.h>

void RTC_init(void)
{
// 1. Select internal 32.768kHz oscillator
RTC.CLKSEL = RTC_CLKSEL_OSC32K_gc;
// 2. Set overflow period: 32768 ticks = 1 second
RTC.PER = 32768;
// 3. Enable overflow interrupt
RTC.INTCTRL = RTC_OVF_bm;
// 4. Enable RTC with no prescaler
RTC.CTRLA = RTC_RTCEN_bm | RTC_PRESCALER_DIV1_gc;
// 5. Enable global interrupts
sei();
}

ISR(RTC_CNT_vect) {
// Toggle LED on PORTB pin 3
PORTB.OUTTGL = PIN3_bm;
// Clear interrupt flag
RTC.INTFLAGS = RTC_OVF_bm;
}

int main(void)
{
    // Set PORTB pin 3 as output
    PORTB.DIRSET = PIN3_bm;

    // Initialize RTC
    RTC_init();

    while (1) {
    // Main loop does nothing
    }
}
```

Lab practice#7: Digital Alarm Clock using RTC

- Students will build a digital alarm clock with the following features:
- Maintain current time using RTC
- Receive time and alarm settings via UART from the user
- Trigger an alarm (on board LED blink) when the set time is reached
- Periodically display current time and alarm status via UART

- 1. UART Command Examples
- SET 12:30:00 → Set current time to 12:30:00
- ALARM 12:35:00 → Set alarm time
- SHOW → Display current time and alarm time
- STOP → Turn off alarm

2. UART Output Example

```
Current Time: 12:34:58
Alarm Time:   12:35:00
Status:       Waiting...
```